D20A **Roll No.19**

Experiment No. 7

Aim:

To establish a local blockchain environment using Ganache and connect it with MetaMask and Remix IDE. To compile, deploy, and interact with a Solidity smart contract using a Ganache account.

Theory:

A smart contract is a blockchain-based program that automatically performs operations according to predefined rules. In this experiment, Ganache is used to create a local Ethereum-compatible blockchain with test accounts, and MetaMask is used to manage an account and connect to the network. Remix IDE provides the environment for writing, compiling, deploying, and interacting with the Solidity contract.

The contract maintains a numerical value and an array of records containing an ID, title, sender address, and timestamp. Functions declared as `view` allow reading blockchain data without executing transactions, whereas functions that modify the contract's state generate transactions, consume gas, and are permanently recorded in blockchain blocks. Events emitted by the contract generate log entries for state-changing operations. Deployment details and transaction records can be verified using the Ganache interface.

Steps:

1. Install and start Ganache to create a local Ethereum-compatible blockchain.
2. Add the Ganache network in MetaMask using the local RPC endpoint and chain ID shown in the environment.
3. Import a Ganache account into MetaMask using its development private key.
4. Open Remix IDE and load Lab7Storage.sol containing the Solidity smart contract.
5. Select the Solidity compiler, compile the contract, and verify that Lab7Storage is compiled successfully.
6. Select the Ganache provider in Remix, choose the compiled Lab7Storage contract, and deploy it using the connected account.
7. Inspect the deployed contract functions in Remix and verify the deployment blocks and transaction details in Ganache.
8. Interact with the contract through its available functions and observe the corresponding blockchain state and transaction information.

Code:

```
// SPDX-License-Identifier: GPL-3.0
/*
 * ========================================================
 * Experiment No : 7
 * AIM                : Introduction to Truffle, Establish local
 *                      blockchain using Truffle/Ganache and
 *                      deployment of Smart Contract
 * Author       : Dipesh Nawani
 * Roll No      : 19
 * Batch        : A
 * NOTE: This is a simple record-keeping contract. Rename the
 * contract/fields to match your chosen Mini Project.
 * ========================================================
 */
```

Experiment No. 7

```
pragma solidity >=0.7.0 <0.9.0;
/// @title Simple record storage (Author: Jay Kerkar | Roll No: 16 | Batch: A)
contract Lab7Storage {
    // Author details stored on-chain as proof of ownership
    string public constant AUTHOR  = "Dipesh Nawani";
    string public constant ROLL_NO = "19";
    string public constant BATCH   = "A";
    // Account that deployed the contract (the Ganache account selected in MetaMask)
    address public owner;
    // A single number that can be stored and retrieved (like Storage.sol)
    uint256 private number;
    // A record entry
    struct Record {
        uint256 id;
        string  title;
        address addedBy;
        uint256 timestamp;
    }
    Record[] private records;
    // Events show up in the transaction logs inside Ganache
    event NumberStored(address indexed by, uint256 value);
    event RecordAdded(uint256 indexed id, string title, address indexed addedBy);
    constructor() {
        owner = msg.sender;
    }
    /// @notice Store a number (this is a transaction -> costs gas, creates a block in Ganache)
    function store(uint256 num) public {
        number = num;
        emit NumberStored(msg.sender, num);
    }
    /// @notice Read the stored number (view call -> free, no block mined)
    function retrieve() public view returns (uint256) {
        return number;
    }
    /// @notice Add a new record (transaction)
    function addRecord(string memory title) public {
        uint256 id = records.length;
        records.push(Record(id, title, msg.sender, block.timestamp));
        emit RecordAdded(id, title, msg.sender);
    }
    /// @notice Get a record by id (view call)
    function getRecord(uint256 id)
        public
        view
        returns (uint256, string memory, address, uint256)
    {
        require(id < records.length, "Record does not exist");
        Record memory r = records[id];
        return (r.id, r.title, r.addedBy, r.timestamp);
    }
```

Experiment No. 7

```
    /// @notice Total number of records (view call)
    function totalRecords() public view returns (uint256) {
        return records.length;
    }
}
```

Output:

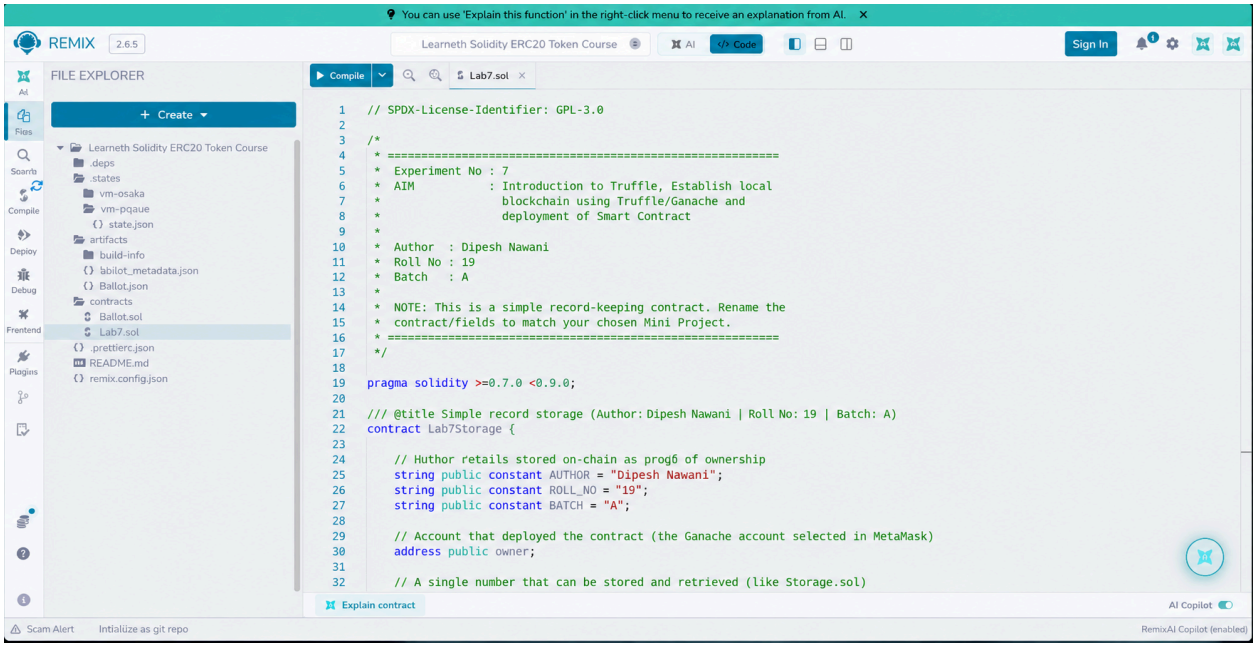


Fig. 1. Solidity contract loaded in Remix IDE

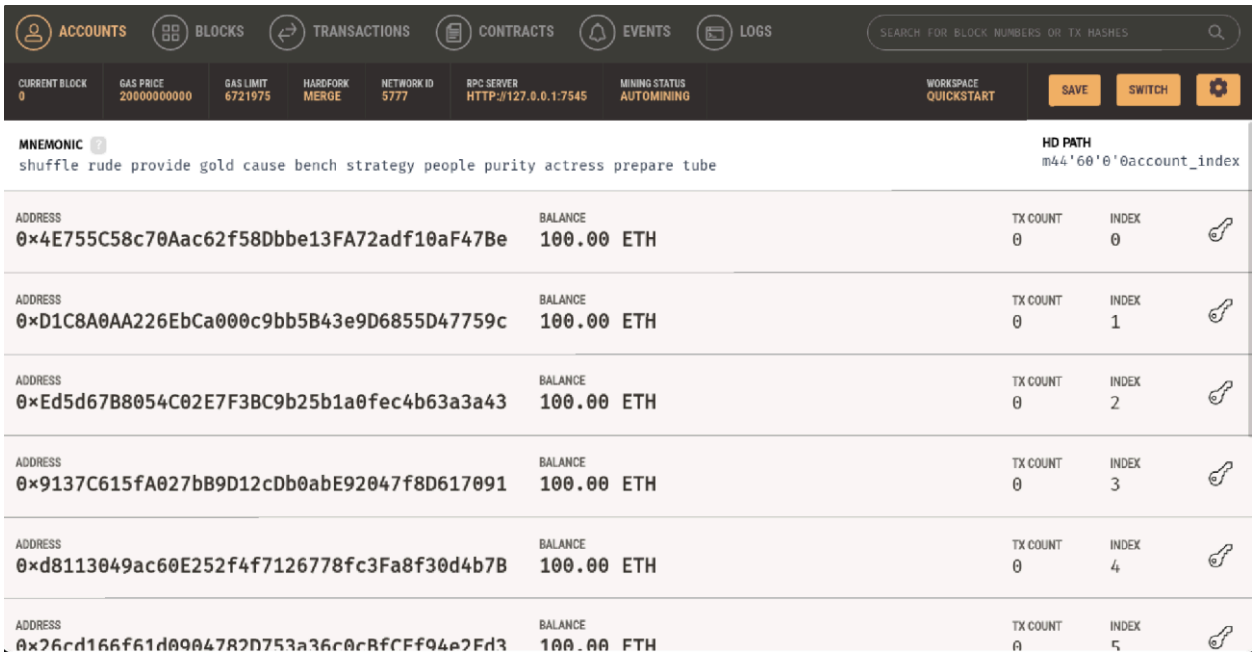


Fig. 2. Ganache development accounts with initial balances

Experiment No. 7

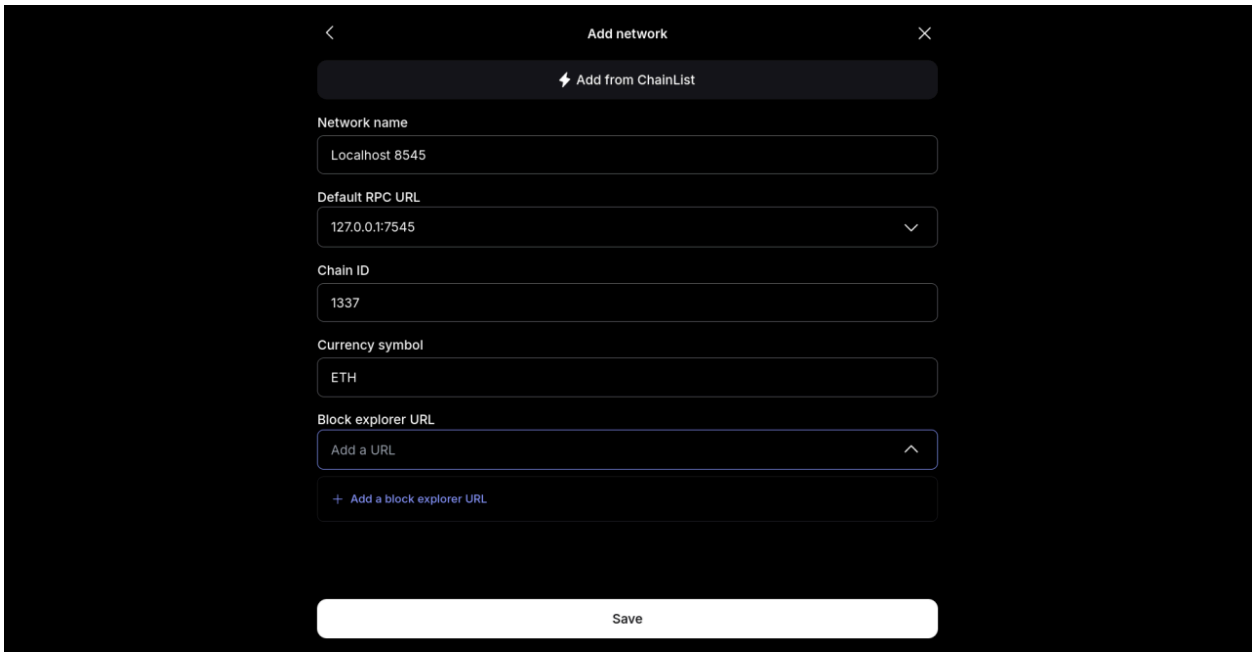


Fig. 3. MetaMask network configured for Localhost 8545



Fig. 4. Ganache account address and development private key

Experiment No. 7

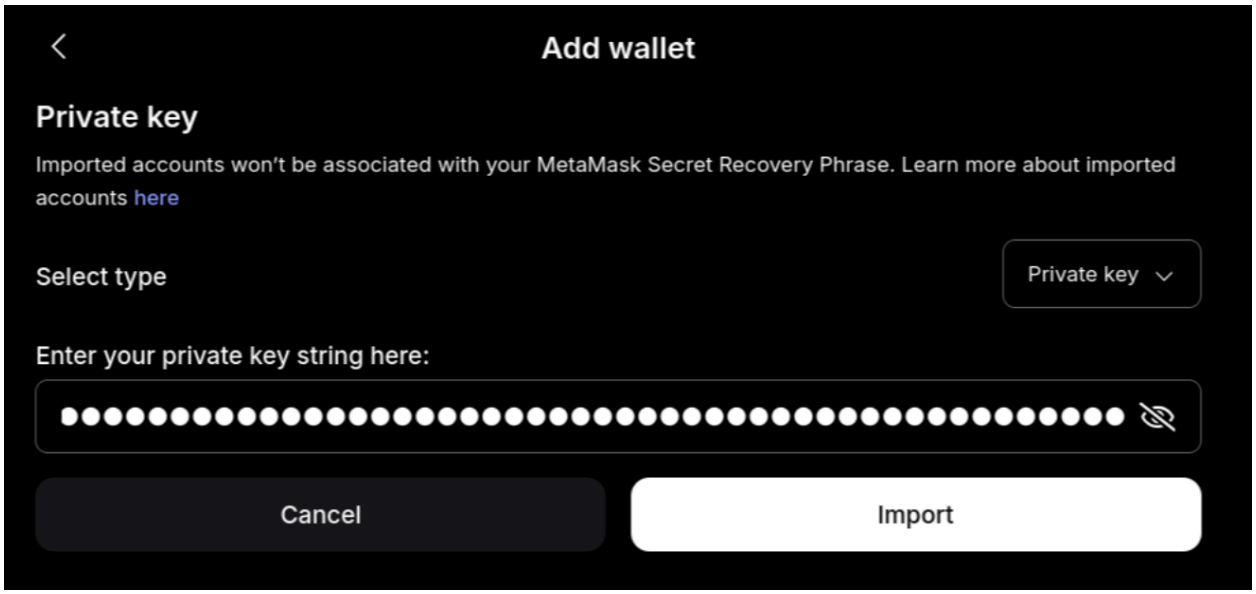


Fig. 5. MetaMask private key import screen

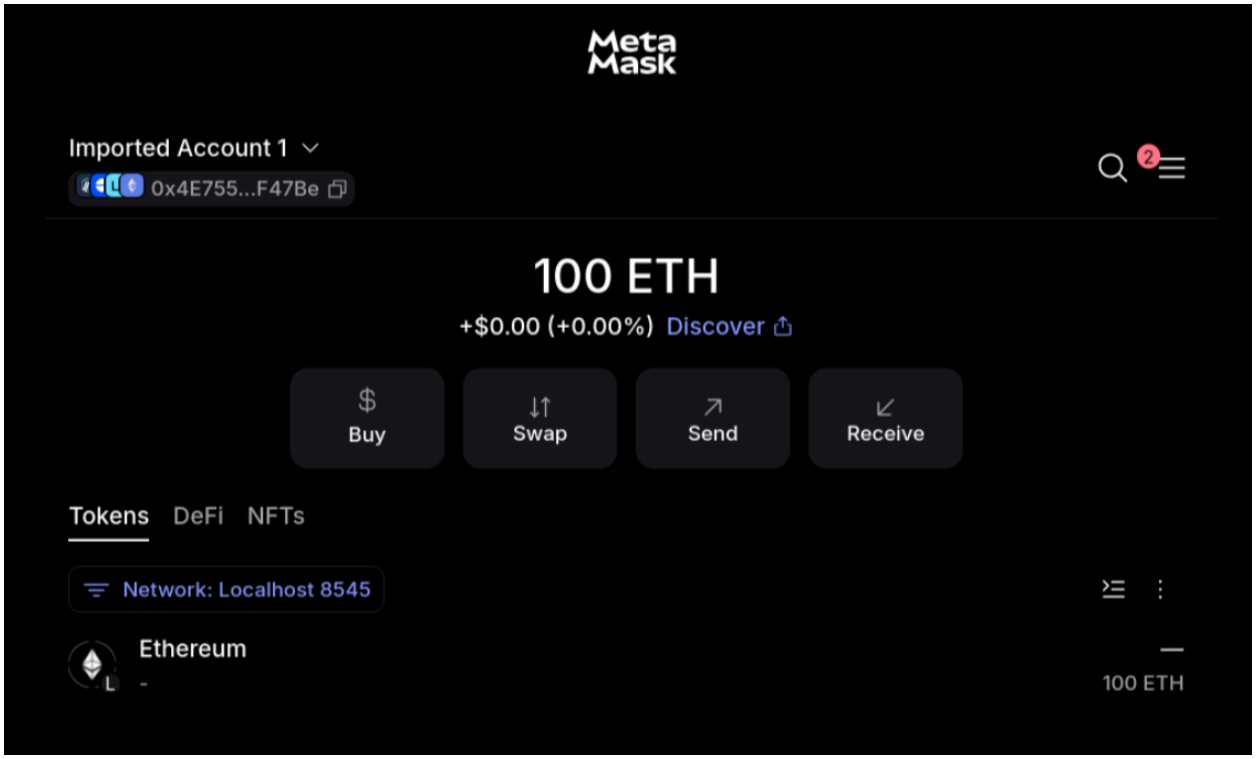


Fig. 6. Imported MetaMask account showing 100 ETH

Experiment No. 7

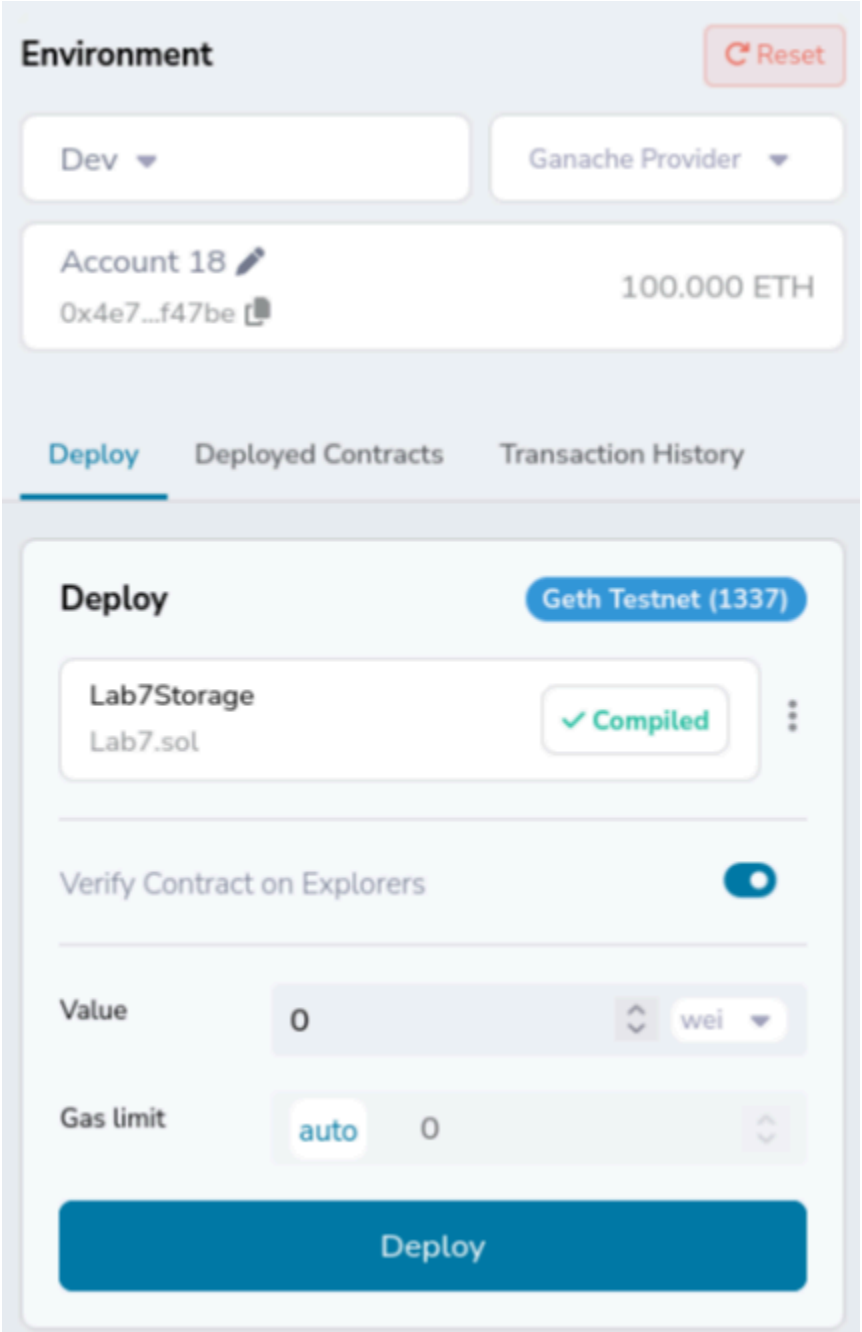


Fig. 7. Remix deployment panel with Ganache provider

Experiment No. 7

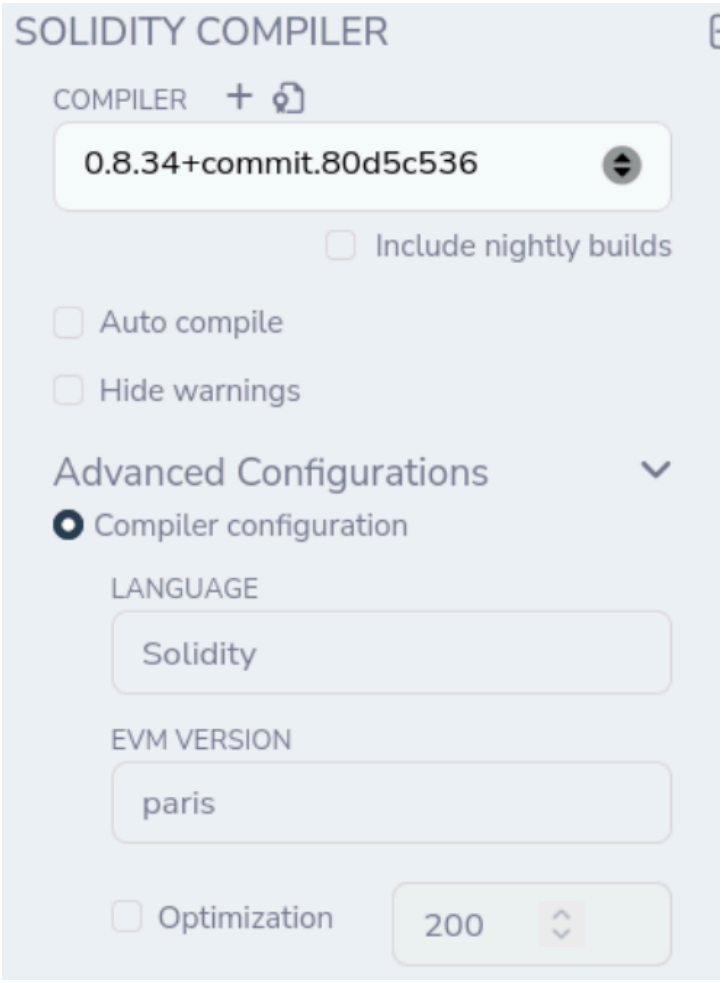


Fig. 8. Solidity compiler configured for contract compilation



Fig. 9. Deployed contract functions visible in Remix

Experiment No. 7

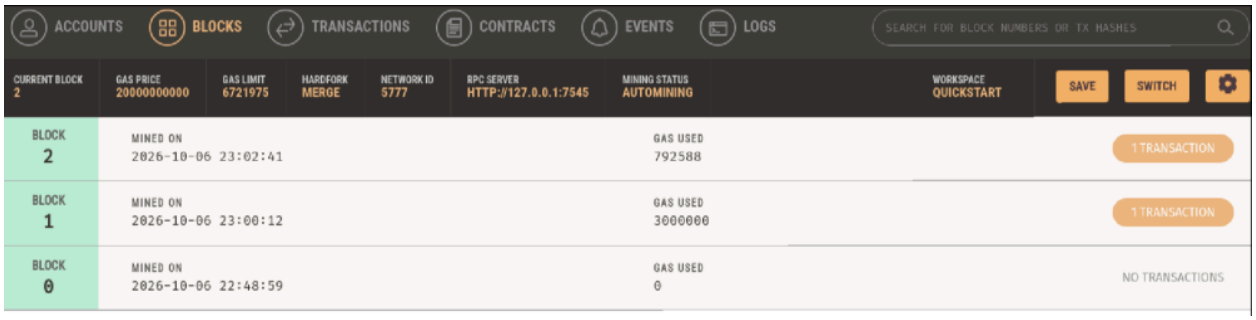


Fig. 10. Ganache blocks showing contract transactions

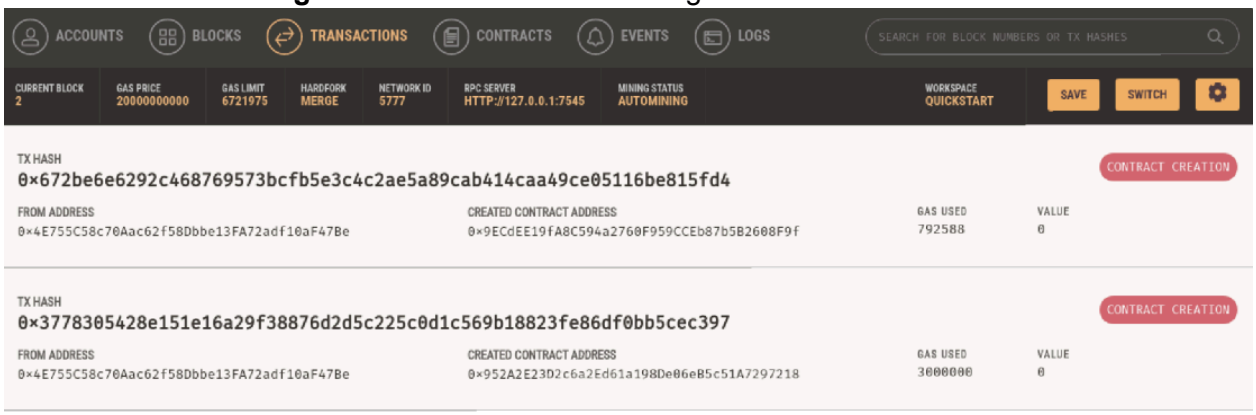


Fig. 11. Ganache transaction list showing contract creation

Conclusion:

Ganache was configured as the local blockchain network and integrated with MetaMask and Remix IDE. The `Lab7Storage` smart contract was successfully compiled and deployed using the Ganache provider, with its available functions visible in Remix. The screenshots captured the local network configuration, connected account, deployed contract, generated blocks, and contract deployment transactions. This experiment demonstrated the fundamental process of setting up a local blockchain, deploying a Solidity smart contract, and interacting with it using development tools.